

Scalable Product Recommendation on Large-Scale Amazon Review Data Using Spark and Alternating Least Squares

Charles Wang
CSC 467

Abstract—Modern marketplaces offer more choice than any shopper can browse, which makes personalized recommendation essential. This paper addresses a single question: given a user’s past product ratings, what should we recommend next? We study the Amazon Reviews 2023 `Video_Games` dataset of 4.62 million reviews. Using Apache Spark, we clean and reduce the raw reviews into a compact ratings table and then build a recommender with Alternating Least Squares (ALS) matrix factorization. After reduction the dataset holds 953,017 ratings. A default ALS model reaches a root-mean-square error (RMSE) of 1.33 on held-out data; tuning the rank and regularization lowers it to 1.28. We pay particular attention to what makes this matrix difficult. It is roughly 99.999% empty and heavily skewed toward five-star ratings, yet this sparsity is not an obstacle but the reason matrix factorization is the right tool.

I. INTRODUCTION AND MOTIVATION

Large online marketplaces list far more products than any user can browse. Amazon carries tens of millions of items, so the platform must predict what each user is likely to value and surface it. The stakes are concrete on every side: sharper recommendations mean more sales for the retailer, less wasted searching for the shopper, and a path to the right buyers for individual sellers. But how do you recommend a product to someone who has rated only a handful out of millions? That is the question this project sets out to answer.

Stated precisely: given a user’s history of product ratings, which additional products should we recommend to that user? We pursue this on the Amazon Reviews 2023 collection released by the McAuley Lab [5]. The full corpus spans 571 million reviews; we narrow to the `Video_Games` category, which holds 4,624,615 reviews and therefore clears the project’s requirement of more than one million entities. The dataset is worth studying for three reasons. It is large-scale, too big to process in memory on one machine, which is why a distributed engine such as Spark is warranted. It is sparse: 2,766,656 users have rated only 137,249 products, leaving a user-by-item matrix that is roughly 99.999% empty. And it is skewed heavily toward positive ratings, which both reflects real user behavior and limits how accurate any model can be.

II. DATA ENGINEERING

A. Acquisition

We downloaded the `Video_Games` review file (≈ 776 MB gzip) and the item metadata file (≈ 98 MB). Each review is a JSON object carrying a user id (`user_id`), a product id

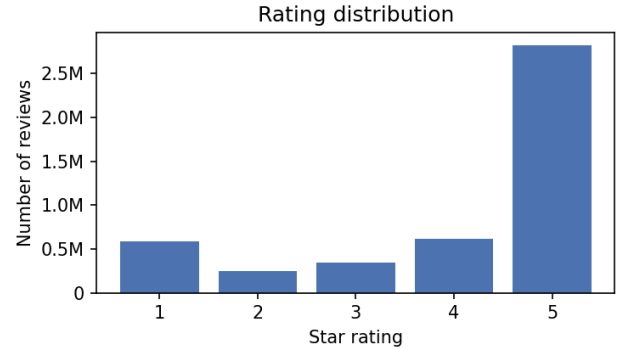


Fig. 1. Rating distribution. More than 60% of ratings are five stars which is a strong positive skew typical of real feedback data.

(`parent_asin`), a 1–5 rating, a timestamp, and free text. The metadata maps each `parent_asin` to a product title; because both files share `parent_asin`, they can later be joined to turn opaque product codes into readable titles.

B. Descriptive Analysis

A first pass over the raw reviews gives the shape of the data: 4,624,615 reviews, 2,766,656 distinct users, 137,249 distinct products, and a matrix sparsity near 0.0012%. The rating distribution (Fig. 1) is the detail that matters most: more than 60% of all ratings are five stars. Together, the sparsity and this skew dictate the modeling decisions in Section III.

C. Cleaning and Reduction

Raw data is rarely the data you need. The reviews carry far more than a recommender requires, so we reduced them through a Spark pipeline. First, we kept only (`user_id`, `parent_asin`, `rating`, `timestamp`) and discarded the review text, the single largest source of shrinkage. Second, we removed null records and collapsed to one rating per (`user`, `item`) pair. Third, we dropped users and products with fewer than five ratings, since a model cannot learn from someone seen only once (the cold-start problem), and this step also strips away the long tail of one-time reviewers. The effect was dramatic: 4,624,615 reviews became 953,017 ratings, just 20.6% of the original (Fig. 2). That 79% reduction is not lost information so much as noise removed—most users had rated only one or two products. We wrote the cleaned table in

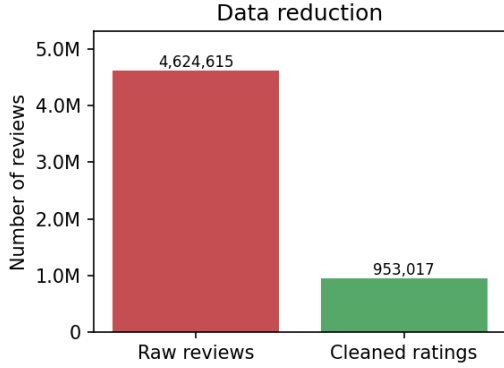


Fig. 2. Data reduction in Task 1: 4.62M raw reviews reduced to 953K cleaned ratings.

Apache Parquet (≈ 30 MB versus the 776 MB source), whose compressed columnar layout makes the analytics that follow noticeably faster.

D. Validation

Inspecting nearly a million rows by eye is impossible, so we verified the cleaned data with assertions that run over the whole table and collapse to a single pass/fail: every rating falls within $[1, 5]$, no field is null, no (user, item) pair is duplicated, and the table is non-empty. All checks passed. One limitation is worth stating plainly: the minimum-ratings filter runs in a single pass, so removing low-count items can push a few users back below five ratings. Guaranteeing a strict minimum would require iterating the filter to a fixed point, the 5-core of the user-item bipartite graph, which we leave as future work rather than claim a guarantee we did not enforce.

III. DATA ANALYTICS

A. Method and Justification

To fill in the blanks of the rating matrix we use collaborative filtering by Alternating Least Squares (ALS), implemented in Spark MLlib. ALS approximates the sparse rating matrix R as a product of two smaller matrices, $R \approx UV^T$, where U holds a short list of latent factors for each user and V holds the same for each item. Intuitively, the algorithm discovers a handful of hidden “preference dimensions” and places every user and every product along them, then predicts a rating by matching the two. The reason this suits our data is precisely the sparsity that looked alarming earlier: ALS learns each factor from only the *observed* ratings and uses them to estimate the unobserved ones. The emptiness of the matrix is not missing data to be repaired—it is the very thing the model exists to predict [1], [4]. ALS also fits Spark naturally, since the two factor matrices can be solved by alternating between them [2].

B. Data Preparation and Tuning

ALS requires integer identifiers, so we mapped `user_id` and `parent_asin` to integer indices with `StringIndexer` and split the ratings 80/20 into train and

TABLE I
ALS TUNING RESULTS (TEST-SET RMSE).

rank	regParam	RMSE
10	0.05	1.43
10	0.10	1.33
50	0.05	1.34
50	0.10	1.28

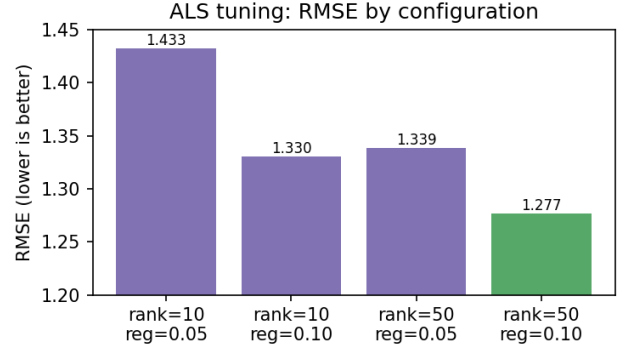


Fig. 3. RMSE by ALS configuration; the best (rank 50, regParam 0.10) is highlighted. A larger rank helps only when paired with enough regularization to prevent overfitting.

test sets under a fixed random seed. Training ran for 10 iterations with nonnegative factors and a cold-start strategy that drops test rows whose user or item was unseen during training. Rather than trust a single guess at the settings, we tuned the two most consequential hyperparameters, the rank (the number of latent factors) and the regularization strength, across a small grid (Table I, Fig. 3).

C. Results

The best configuration (rank 50 with regularization 0.10) reached an RMSE of 1.28, improving on the 1.33 default baseline. The grid is also instructive on its own: a higher rank pays off only when the regularization is strong enough to keep it from overfitting, which is why rank 50 at the weaker setting actually trails rank 10. For a sample of users we generated top-10 recommendations and joined the metadata to recover titles. Two patterns stood out. Predicted scores sometimes climb past the 1–5 range, because ALS is an unbounded regression over latent factors; this is harmless, since only the *ranking* of recommendations is used. More tellingly, a small set of products (a popular controller accessory and several older casual games) surfaced for nearly every user, and the lists leaned toward generic accessories rather than personalized picks. This is popularity bias, the familiar failure mode of collaborative filtering: when the model knows little about a particular user, it falls back on what is popular overall. It is the same sparsity from Section II showing its other face.

IV. RELATED WORK

Collaborative filtering has a long lineage. Sarwar *et al.* framed it around item-item similarity rather than user-user

similarity [6]. Matrix factorization came to prominence during the Netflix Prize, where Koren *et al.* surveyed the latent-factor methods that proved most effective [1], and Zhou *et al.* introduced the large-scale parallel ALS we rely on here [4]. Hu *et al.* adapted these ideas to implicit-feedback data [3], a promising route for ratings as skewed as ours. On the systems side, Zaharia *et al.* describe Apache Spark [2], and Hou *et al.* document the dataset [5]. Our contribution is to adapt parallel ALS on Spark to a single large Amazon category, with an emphasis on the data-engineering reduction and on reporting baseline quality honestly rather than flatteringly.

V. CONCLUSION

We set out to recommend products from a user’s rating history at scale. Using Spark we reduced 4.62 million raw reviews to a validated 953,017-rating table (a 79% cut driven by the data’s own sparsity) and trained a tuned ALS model that reached an RMSE of 1.28. Beyond the single number, the project illustrates the shape of big-data work: acquire data too large for one machine, engineer it into a compact and trustworthy form, and let a distributed algorithm surface the pattern inside. The central insight is an inversion of intuition: the near-empty rating matrix that first looks like a defect is what makes matrix factorization worth doing, while the same sparsity that motivates the method also caps how far it can go. Future work includes iterating the k-core filter toward a denser matrix, widening the hyperparameter search, and exploring implicit-feedback models better matched to skewed ratings.

USE OF AI ASSISTANCE

In the interest of transparency, I have used AI tools such as ChatGPT in the creation of this PDF on Overleaf. I asked it to process my writings, as well as suggesting grammatical checks and sentence structures, in a format that is parsable with LaTeX. Proper formatting for bibliography was also used with AI assistance. I also used Codex assistance for debugging. I take full responsibility for the content and I confirm that every reported result was produced by my project’s own code.

REFERENCES

- [1] Y. Koren, R. Bell, and C. Volinsky, “Matrix factorization techniques for recommender systems,” *Computer*, vol. 42, no. 8, pp. 30–37, 2009.
- [2] M. Zaharia *et al.*, “Apache Spark: a unified engine for big data processing,” *Commun. ACM*, vol. 59, no. 11, pp. 56–65, 2016.
- [3] Y. Hu, Y. Koren, and C. Volinsky, “Collaborative filtering for implicit feedback datasets,” in *Proc. IEEE ICDM*, 2008, pp. 263–272.
- [4] Y. Zhou, D. Wilkinson, R. Schreiber, and R. Pan, “Large-scale parallel collaborative filtering for the Netflix prize,” in *Proc. AAIM*, 2008, pp. 337–348.
- [5] Y. Hou, J. Li, Z. He, A. Yan, X. Chen, and J. McAuley, “Bridging language and items for retrieval and recommendation,” arXiv:2403.03952, 2024.
- [6] B. Sarwar, G. Karypis, J. Konstan, and J. Riedl, “Item-based collaborative filtering recommendation algorithms,” in *Proc. WWW*, 2001, pp. 285–295.